الأكاديمية العربية للعلوم والتكنولوجيا والنقل البحري

Arab Academy for Science, Technology & Maritime Transport

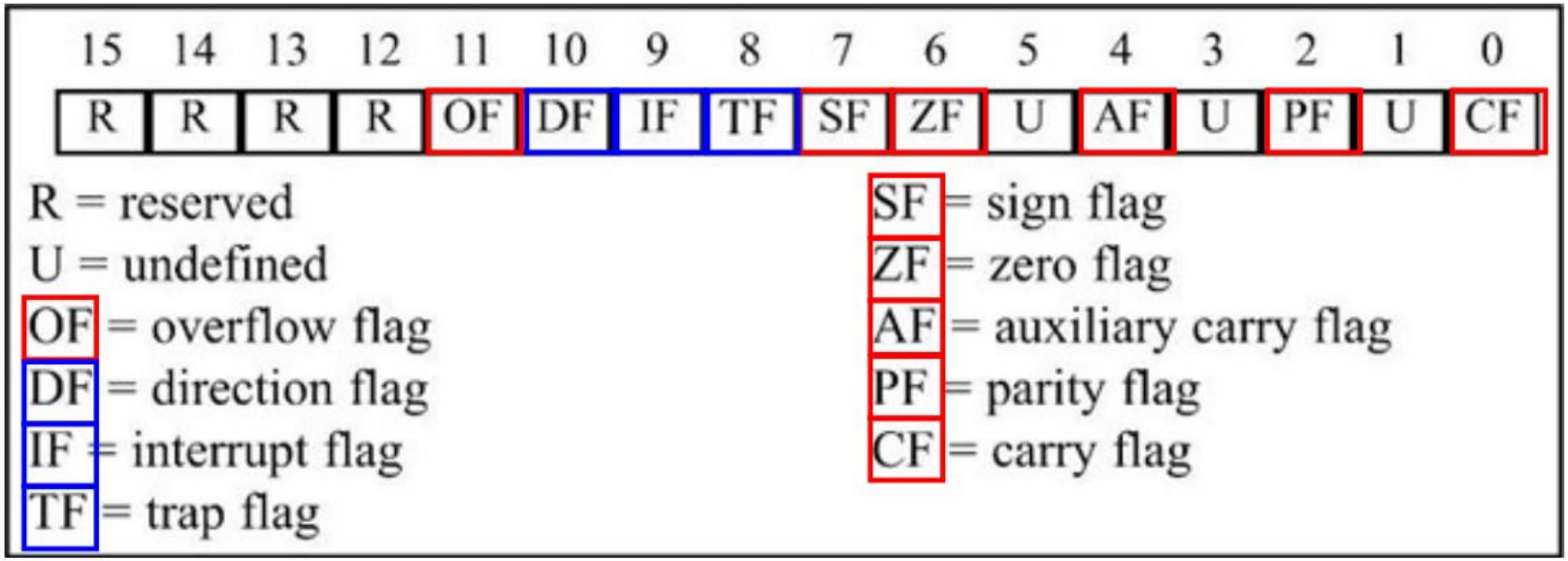
Lecture 3

Flag Register

Overview:

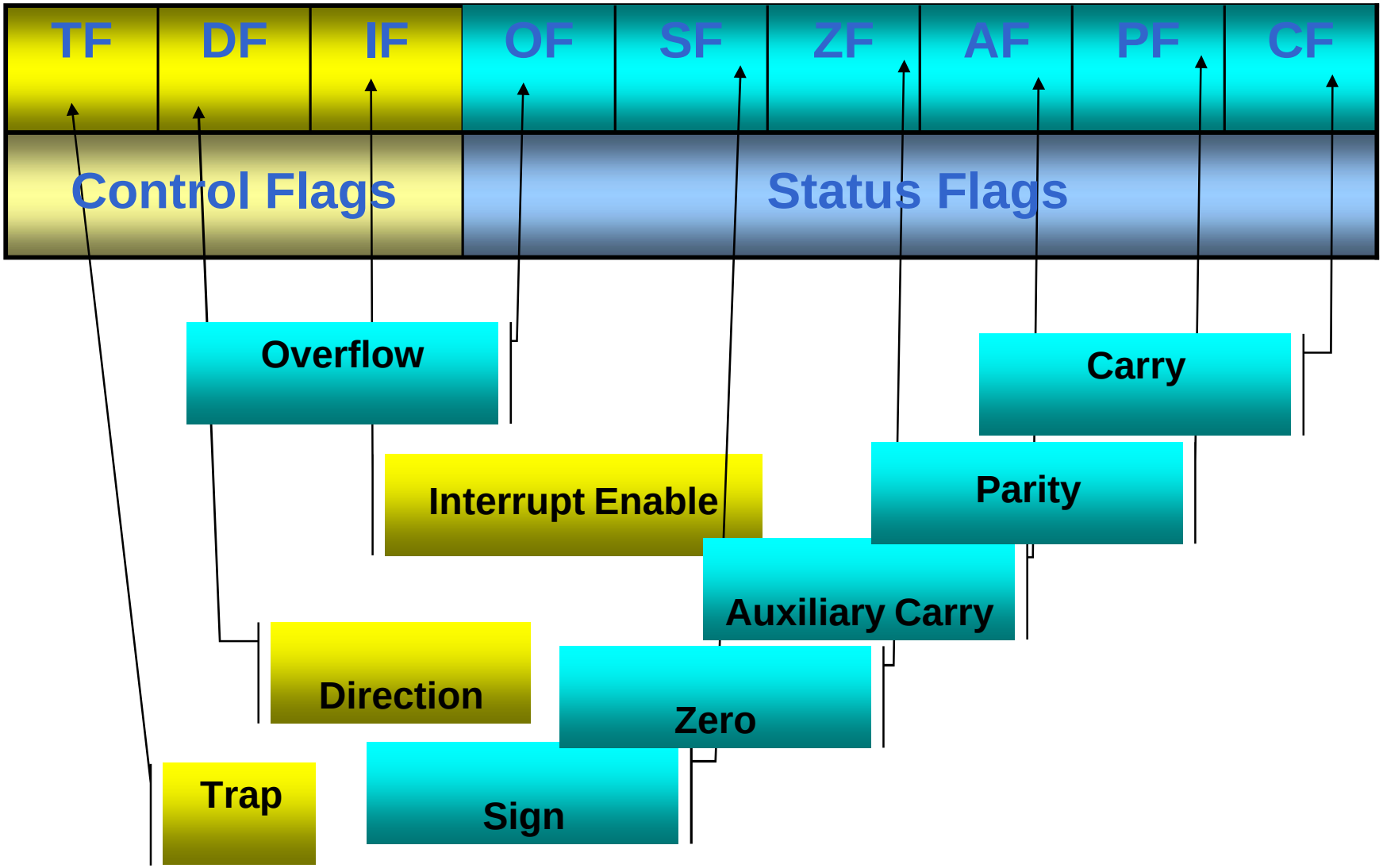
The flag register in a microprocessor is a special-purpose register used to indicate the status of various operations performed by the processor. It contains several individual flags that are either control flags or status flags, each representing specific conditions in the processor's operations.

- **Six flags, called *conditional flags*, indicate some condition resulting after an instruction executes.**

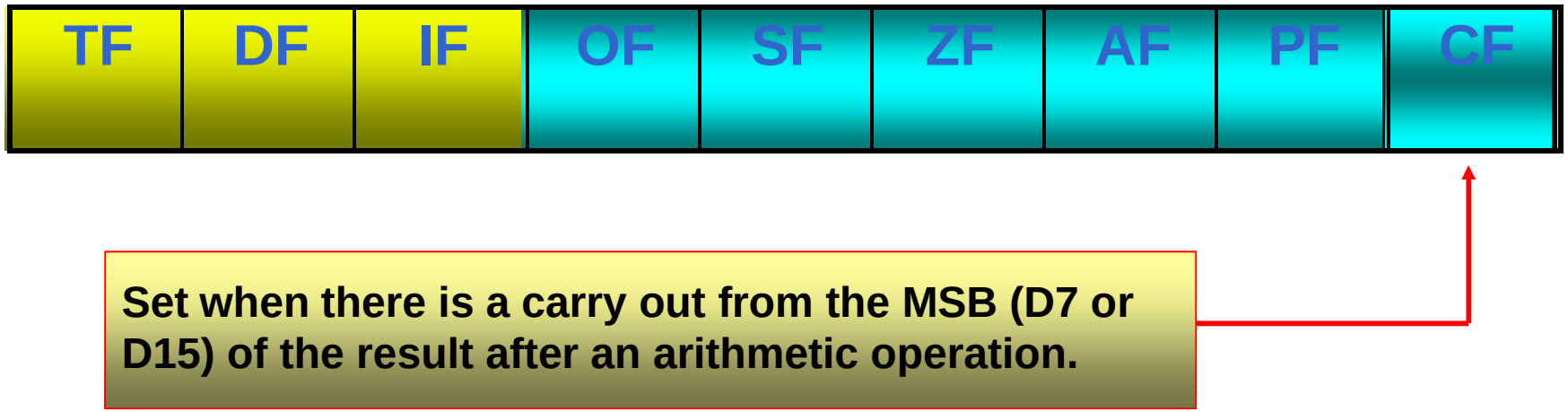


- **These six are CF, PF, AF, ZF, SF, and OF.**
- **The remaining three, often referred to as control flags, control the operation of instructions before execution.**

Status and Flag Register



Status and Flag Register



Example of CF in Arithmetic Operations

When adding two large unsigned integers, if the result exceeds the storage capacity, the CF is set.

```
MOV BH, 38H ; Load 38H into BH
ADD BH, 2FH ; Add 2F to BH, BH becomes 67
```

- CF = 0

```
MOV AX, 0FFFFh; Load AX with FFFFh (65535 in decimal)
MOV BX, 0001h ; Load BX with 0001h (1 in decimal)
ADD AX, BX     ; Add BX to AX
```

- CF = 1

Status and Flag Register

Set if the low-order byte of the result contains an even number of 1s.

Example of PF in Arithmetic Operations

Commonly used in error-checking operations where parity is a factor.

```
MOV BH, 38H ; Load 38H into BH
ADD BH, 2FH ; Add 2F to BH, BH becomes 67
```

- PF = 0

```
MOV AX, 94C2H   ; Load 94C2H into AX
MOV BX, 323EH   ; Load 323EH into BX
ADD AX, BX      ; Add BX to AX; Result = C700H (hex) = 1100 0111 0000 0000 (binary)
```

- PF = 1

Addition:

$$\mathrm{AX} + \mathrm{BX} = 94C2H + 323EH = C700H$$

The lower byte contains an even number of set bits, resulting in the Parity Flag (PF) being set to 1.

Set if there is a carry out from bit 3 to bit 4, often used in BCD operations.

```
MOV AL, 9Ch      ; Move hexadecimal value 9C into AL register
MOV DH, 64h      ; Move hexadecimal value 64 into DH register
ADD AL, DH       ; Add the value in DH to AL
```

Solution:

```
   9C     1001 1100
 + 64     0110 0100
 -------- 0000 0000
```

- **CF = 1** since there is a carry beyond d7
- **AF = 1** since there is a carry from d3 to d4 → (Carry from the lower nibble)
- **PF = 1** since there is an even number of 1s in the result

Status and Flag Register

Set when the result of an arithmetic or logical operation equals zero.

```
MOV AL, 9Ch        ; Move hexadecimal value 9C into AL register
MOV DH, 64h        ; Move hexadecimal value 64 into DH register
ADD AL, DH         ; Add the value in DH to AL
```

ADD AL, DH:

- **AL = AL + DH = 9C + 64 = 156 + 100 = 256 (decimal).**
- In hexadecimal, the result is `100h`.
- **Result in AL = 00h (binary: 00000000).**
  - **ZF = 1** (zero flag is set).

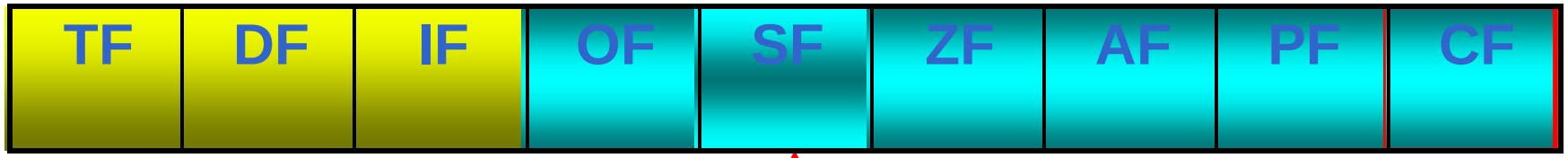
```
MOV AX, 94C2H    ; Load 94C2H into AX
MOV BX, 323EH    ; Load 323EH into BX
ADD AX, BX       ; Add BX to AX; Result = C700H (hex) = 1100 0111 0000 0000 (binary)
```

Conversion to Binary

- **AX:** `94C2H` = `1001010011000010`
- **BX:** `323EH` = `0011001000111110`
- **Result:** `C700H` = `1100011100000000`

- **CF:** 0
- **AF:** 1
- **PF:** 1
- **ZF:** 0

Status and Flag Register



After arithmetic or logical operations,the status of the sign bit(the MSB) is stored in the SF, which is set to 1 for negative results and reset to 0 for positive results.

```
MOV AL, 9Ch       ; Move hexadecimal value 9C into AL register
MOV DH, 64h       ; Move hexadecimal value 64 into DH register
ADD AL, DH        ; Add the value in DH to AL
```

Solution:

```
   9C     1001 1100
+ 64      0110 0100
-------- 0000 0000
```

Addition Result

- **Result:** `100H` = `00000001 00000000`

- **CF**: 1
- **AF**: 1
- **PF**: 0
- **ZF**: 0
- **SF** : 0

```
MOV AX, 94C2H   ; Load 94C2H into AX
MOV BX, 323EH   ; Load 323EH into BX
ADD AX, BX      ; Add BX to AX; Result = C700H (hex) = 1100 0111 0000 0000 (binary)
```

Conversion to Binary

- **AX:** `94C2H` = `1001010011000010`
- **BX:** `323EH` = `0011001000111110`
- **Result:** `C700H` = `1100011100000000`

- **CF**: 0
- **AF**: 1
- **PF**: 1
- **ZF**: 0
- **SF** :1

Status and Flag Register

Set whenever the result of a signed number operation exceeds the capacity, causing an overflow into the sign bit.

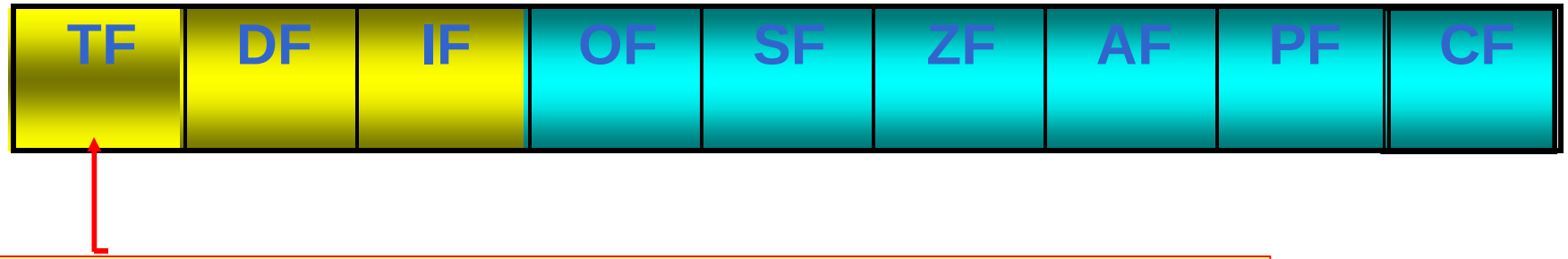
Example 1: Adding Bytes

```
MOV AL, 7FH            ; Move 7FH (01111111) into AL
ADD AL, 02H            ; Add 02H (00000010) to AL
; The result is 81H (10000001), which exceeds the maximum signed byte value (7F); OF = 1
```

Example 2: Adding Words

```
MOV AX, 7FFFH          ; Move 7FFFH (0111111111111111) into AX
ADD AX, 0001H          ; Add 0001H (0000000000000001) to AX
; The result is 8000H (1000000000000000), causing an overflow since it flips to negative;OF = 1
```

Status and Flag Register

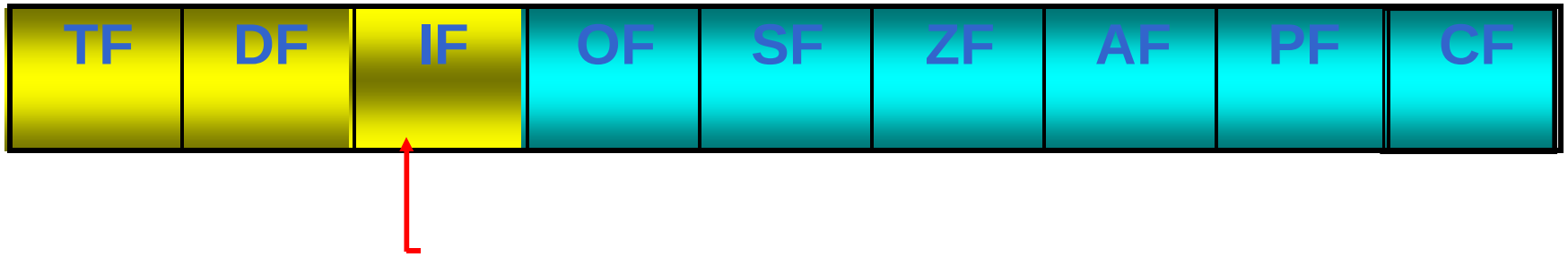


When set, it allows the program to execute in single-stepmode.

In **single-step mode**:

- **Interpreter**: The program executes one line at a time, allowing you to see each result immediately.

Status and Flag Register



When set, it enables external maskable interrupts.

Example:

When the Interrupt Flag is set, the system can respond to external interrupts, like a keyboard or timer interrupt. If the flag is cleared, these interrupts are ignored until it's set again.

Status and Flag Register

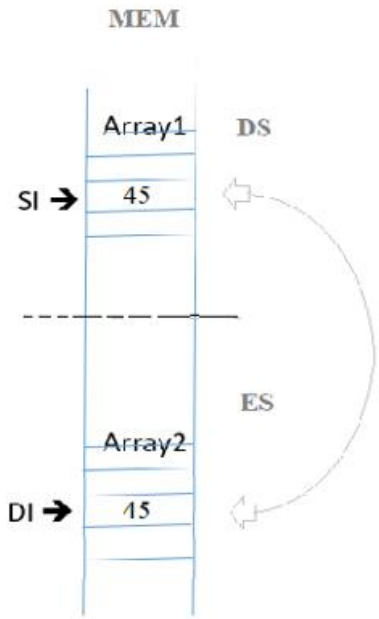
TF	DF	IF	OF	SF	ZF	AF	PF	CF

Controls the direction of string operations. When set, the pointers are automatically decremented.

Example:

When the Direction Flag is set, during string operations like **MOVS,** the Source Index (SI) and Destination Index (DI) are decremented after each operation, allowing the string to be processed from higher memory addresses to lower memory addresses.

Using **CLD** clears the Direction Flag, causing SI and DI to increment, while **STD** sets the flag, making SI and DI decrement during string operations.



ADD Instruction: Example 1

```
MOV    BH,38H        ; (BH) ← 38H
ADD    BH,2FH        ; ADD 2F to (BH) ;now (BH)← 67H
```

38

2F

67

- **Result**: `01100111` (67H in hexadecimal)

CF=0	**PF=0**
AF=1	**ZF=0**
SF=0	**OF=0**

ADD Instruction: Example 2

```
MOV   AL,9CH      ; AL=9CH
MOV   DH,64H      ; DH=64H
ADD   AL, DH      ; ADD DH to AL ;now AL=00
```

9C

64

00

- **Result:** `100000000` (100H in hexadecimal, which exceeds 8 bits)

CF=1	**PF=1**
AF=1	**ZF=1**
SF=0	**OF=0**

ADD Instruction: Example 3

```
MOV     AX,34F5H        ; AX=34F5
ADD     AX,95EBH        ; now AX=CAE0
```

35F5

95EB

CAE0

- Result = `1100 1010 1110 0000` (in hexadecimal: `CAE0H`)

CF=0	**PF=0**
AF=1	**ZF=0**
SF=1	**OF=0**

ADD Instruction: Example 4

```
MOV     BX,0AAAAH       ; BX=AAAA
ADD     BX,5556H        ; now BX=0000
```

AAAA

5556

0000

```
   1010 1010 1010 1010
+  0101 0101 0101 0110
-----------------------
   0000 0000 0000 0000 (with a carry of 1)
```

CF=1	**PF=1**
AF=1	**ZF=1**
SF=0	**OF=0**

ADD Instruction: Example 5

- **Write an assembly code that adds the two signed numbers, +96 and +70, and show how the flag registers (the six flags) are affected by this addition.**

```
MOV   AL,60H      ; AL=0110 0000 (+90)
MOV   BL,46H      ; BL=0100 0110 (+60)
ADD   AL, BL      0110 0000
                  0100 0110
                  1010 0110
```

CF=0	**PF=1**
AF=0	**ZF=0**
SF=1	**OF=1**